

Exercise 2: Store Your Data

Interfaces 102: Building Dynamic Interfaces

Introduction	2
How Can I Practice?	2
Appian Version	2
Naming Conventions	2
Save Often	2
Additional Resources	2
Generate the Create Estimate Record Action	3
Store Query Data in a Local Variable	4
Create a Local Variable	5
Query the Maintenance Record Type	5
Save Your Query as an Expression Rule	7

Notice of Rights

This document was created by Appian Corporation, 7950 Jones Branch Dr, Tysons, Virginia 22102. Copyright 2026 by Appian Corporation. All rights reserved. Information in this document is subject to change. No part of this document may be reproduced or transmitted in any form by any means, without prior written permission of Appian Corporation. For more information on obtaining permission for reprints or excerpts, contact Appian Training at academyonline@appian.com.

Introduction

How Can I Practice?

You should practice in Appian Community Edition. Keep in mind that this is a community environment not suitable for production workloads or sensitive information.

Appian Version

Appian Community Edition is on the latest Appian version. If you are following the exercises from a previous Appian version, go to academy.appian.com to download the latest version.

Naming Conventions

When you register for Appian Community Edition, you gain access to a workspace, which is your personal area within the community environment. When your workspace has been assigned, you will receive a confirmation email with a workspace ID. Pre-loaded apps in your workspace include the workspace ID.

Use this workspace ID when creating new applications and objects. Otherwise, you could run into naming conflicts with objects created by other users. In the example below, the application prefix for the Acme Auto Solution app is W0000AS. So, the workspace ID is W0000, which would be included in any new app's prefix.

Save Often

Appian does not automatically save updates, so save your objects frequently.

Additional Resources

Appian provides a number of training resources for Appian developers. The following resources are particularly popular with our learners:

- [Academy Online](#) - Appian's online courses provide useful survey courses, step-by-step tutorials, and practice exercises. Explore these resources at your own pace. Survey courses will help you develop a better grasp of the topics you need to learn. Video and print tutorials will help you with getting hands-on experience with Appian.
- [Community Discussions for New Users](#) - Check out the **New to Appian** thread in Community. Join our community of experts to ask questions and find answers from past discussions.
- [Appian Documentation](#) - Appian's product documentation will provide you with an overview of key Appian features, newest release information, additional tutorials, and helpful patterns and recipes to implement in your app.

Generate the Create Estimate Record Action

In this exercise, you will generate a record action to create a new estimate. As part of this process, Appian will create an interface for you. In this, and subsequent, exercises, you will customize this generated interface. Follow the steps below to create the record action.

1. Open the **W#ADX Estimate** record type and navigate to the **Actions** page.
2. Click **GENERATE RECORD ACTIONS**.
3. In the dialog, unselect the **Update** and **Delete** options, then click **NEXT**.

What actions would you like to create?

 Create Add new records and related records	 Update Update a specific record and its related data	 Delete Delete a specific record and its related data
---	---	---

4. Under **Relationships**, select **part**. Then, click **NEXT**.

Select Record Types

Select one or more record types to include on the form.

[Deselect all record types](#)

Record Type

We'll generate a section of input, selection, or picker components based on the fields in your record type, including any many-to-one relationships

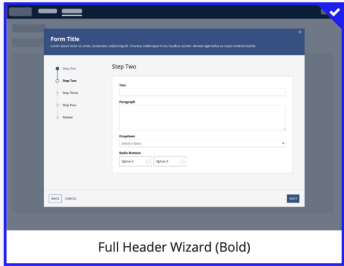
☒ Estimate

Relationships

We'll choose the best input style for each relationship by adding a section of inputs, an editable grid, or an input component, based on the relationship type

☒ part

5. Leave the recommended template selected, and click **GENERATE**.



Full Header Wizard (Bold)

Recommended Template

Based on the number of record types you selected, we recommend this template for the best user experience. If your data is simple and doesn't need to be broken into separate steps, consider choosing a single-step form template.

6. Once complete, you will see the two new objects created in the **Build** view.

☐	Name	Description
☐	W5656ADX Create Estimate	Create a new Estimate
☐	W5656ADX_CreateEstimate	Form interface for creating new Estimates as an action
☐	W5656ADX Estimate	Estimate for vehicle maintenance
☐	W5656ADX Part	Parts related to a vehicle maintenance estimate.

It is the interface **W#ADX_CreateEstimate** that you will customize in this course.

Store Query Data in a Local Variable

The generated interface uses an integer field to allow the user to establish a relationship between the estimate and the maintenance request.

Create Estimate

Enter details for the estimate

☐ Details

☐ Part

Details

Maintenance Id *

CANCEL

NEXT

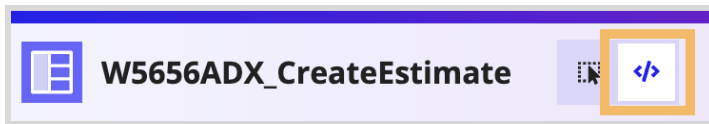
Although this is functional (as long as the user knows the id of the maintenance request), it is not particularly convenient. One of the ways you will update this interface is to replace this field

with a list of cards that display vehicle and maintenance request information. The user will then be able to select an available maintenance request from this list of cards.

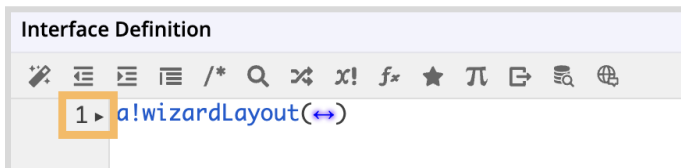
To do this, you will first need to write a query that returns the open maintenance requests and stores them in the interface.

Create a Local Variable

1. Switch to **Expression Mode**.

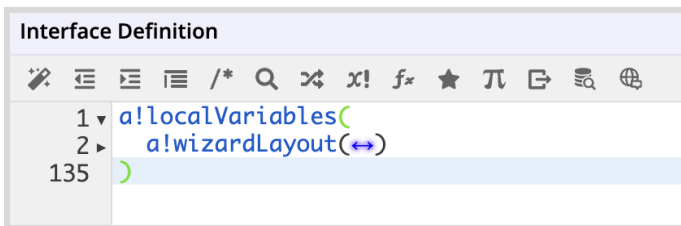


2. Click the arrow by the line number for the first line in the expression. This will collapse the **a!wizardLayout()**.



Collapsing sections of the interface expression can make working with long expressions less cumbersome and reduce the chance of making errors.

3. Wrap the **a!wizardLayout()** in an **a!localVariables()** function.



4. Before the **a!wizardLayout()**, add a new line and create the local variable **local!availableMaintenanceRequests**.

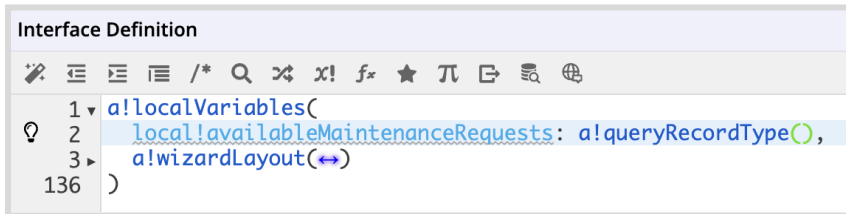


You must add a comma after the declaration of the local variable. If you do not, your interface will display an error.

Query the Maintenance Record Type

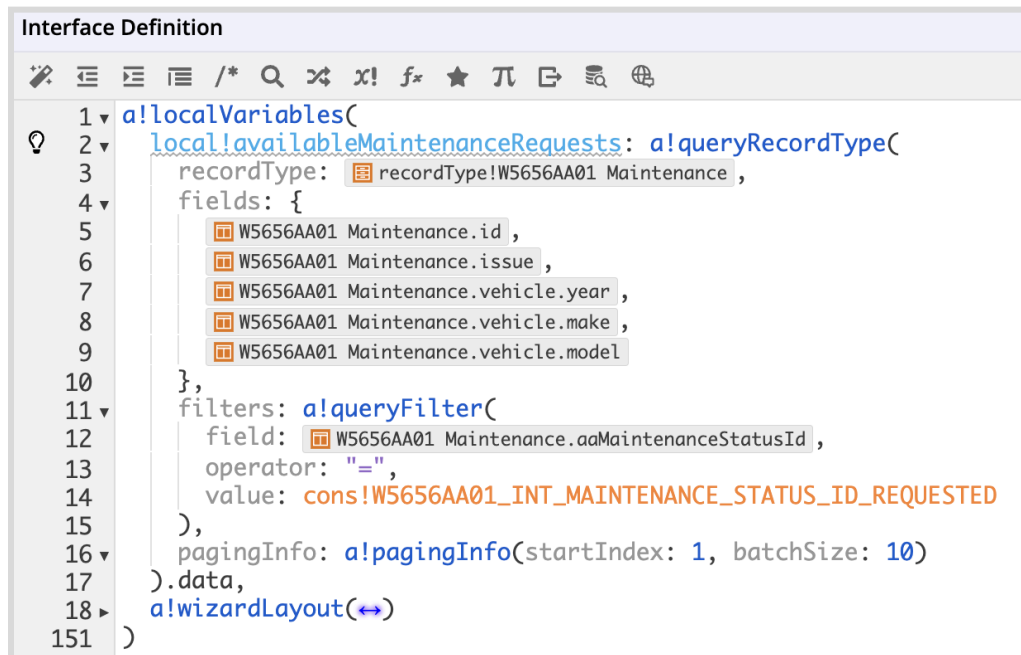
You've created a local variable to store the maintenance requests your interface will use, but, currently, that local variable has no data. Now you will create a query to return a subset of the maintenance requests to store to the local variable.

1. Add a colon after the local variable name (before the comma and add **a!queryRecordType()**.



```
Interface Definition
1  a!localVariables(
2  local!availableMaintenanceRequests: a!queryRecordType(),
3  a!wizardLayout(↔)
136 )
```

2. Configure the query with the following settings. Feel free to type them manually, or to use the **Query Editor** by clicking the icon  in the toolbar.
 - a. **recordType:** W#AA Maintenance
 - b. **fields:**
 - W#AA Maintenance.id
 - W#AA Maintenance.issue
 - W#AA Maintenance.vehicle.year
 - W#AA Maintenance.vehicle.make
 - W#AA Maintenance.vehicle.model
 - W#AA Maintenance.vehicle.image
 - c. **filters:** W#AA Maintenance.aaMaintenanceStatusId = cons!W#AA_INT_MAINTENANCE_STATUS_ID_REQUESTED
 - d. **pagingInfo:** a!pagingInfo(startIndex: 1, batchSize:10)
 - e. Be sure your query ends with **.data**. This will return just the data of the query and not metadata like error codes.

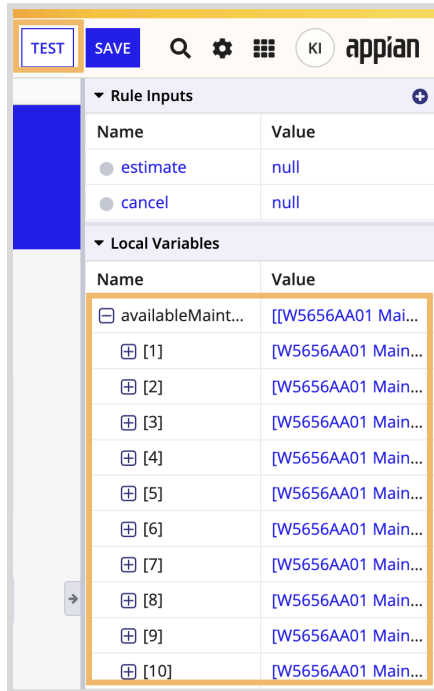


```
Interface Definition
1  a!localVariables(
2  local!availableMaintenanceRequests: a!queryRecordType(
3    recordType: recordType!W5656AA01 Maintenance ,
4    fields: {
5      W5656AA01 Maintenance.id ,
6      W5656AA01 Maintenance.issue ,
7      W5656AA01 Maintenance.vehicle.year ,
8      W5656AA01 Maintenance.vehicle.make ,
9      W5656AA01 Maintenance.vehicle.model
10   },
11   filters: a!queryFilter(
12     field: W5656AA01 Maintenance.aaMaintenanceStatusId ,
13     operator: "=",
14     value: cons!W5656AA01_INT_MAINTENANCE_STATUS_ID_REQUESTED
15   ),
16   pagingInfo: a!pagingInfo(startIndex: 1, batchSize: 10)
17 ).data,
18 a!wizardLayout(↔)
151 )
```

Notice that the filter uses a constant to identify only maintenance requests with the status "Requested" and that the batchSize is limited to 10. This is just to reduce the number of maintenance requests shown on screen in this example.

Your interface expression may look slightly different than the image above if you used the Query Editor.

3. Click **TEST**, then **TEST INTERFACE** to update the value of the local variable. Then check its value in the Local Variables pane. You should see a list of 10 W#AA Maintenance records.

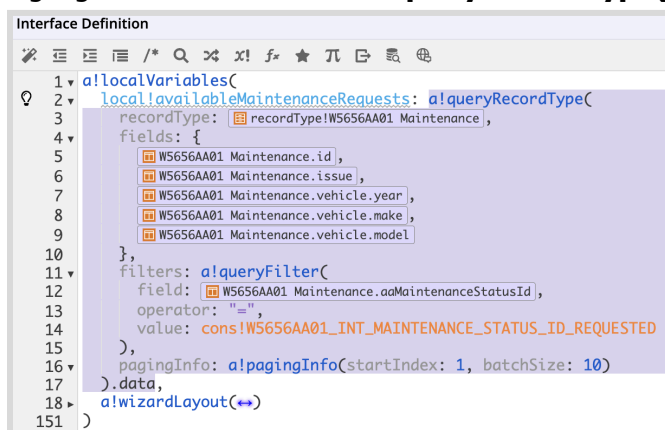


You now have the maintenance request info you need stored in a local variable that you can reference elsewhere in your interface expression in later exercises.

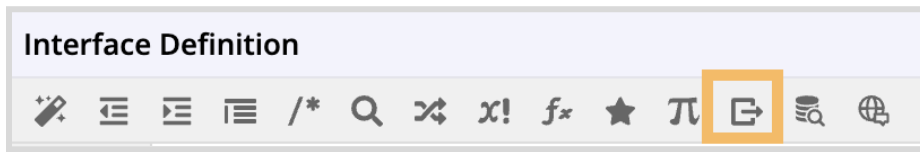
Save Your Query as an Expression Rule

It is best practice to save queries like this in expression rules instead of having them hard-coded into an interface. If there is a chance that this query might be useful elsewhere in the application, saving it as an expression rule makes it easy to reuse.

1. Highlight the full text of the **a!queryRecordType()** function.



2. Click the **Save Selected Expression As...** icon in the toolbar.



3. In the dialog configure the following:
 - **Name:** Enter `W#ADX_getRequestedMaintenance`
 - **Description:** Enter `Returns maintenance and vehicle information for maintenance request with status REQUESTED.`

Save Expression As

Save As *
Expression Rule Interface

Name *
W5656ADX_getRequestedMaintenance

Description
Returns maintenance and vehicle information for maintenance request with status REQUESTED.

Application *
Appian Developer Sandbox

Save In *
ADX Rules & Constants

Create New Rule Folder

CANCEL SAVE

Expression Preview

```
1 queryRecordType(  
2   recordType: recordType!W5656AA01 Maintenance ,  
3   fields: {  
4     W5656AA01 Maintenance.id ,  
5     W5656AA01 Maintenance.issue ,  
6     W5656AA01 Maintenance.vehicle.year ,  
7     W5656AA01 Maintenance.vehicle.make ,  
8     W5656AA01 Maintenance.vehicle.model  
9   },  
10  filters: queryFilter(  
11    field: W5656AA01 Maintenance.adMaintenanceStatusId ,  
12    operator: "=",  
13    value: cons!W5656AA01_INT_MAINTENANCE_STATUS_ID_REQUESTED  
14  ),  
15  pagingInfo: apagingInfo(  
16    startIndex: 1,
```

4. Click **SAVE**.
5. Be sure to save your interface.